

Testing a Streamlit App

Part of the [Streamlit curriculum](#). This document covers Streamlit's `AppTest` framework, and a candid discussion of when manual verification is a reasonable alternative.

1. Streamlit's headless testing utility

Streamlit ships `streamlit.testing.v1.AppTest`, which runs a script without a browser and lets you assert against the resulting widget tree:

```
from streamlit.testing.v1 import AppTest

def test_overview_page_shows_metric():
    at = AppTest.from_file("app.py").run()
    assert at.metric[0].value == "1,234"
    assert not at.exception

def test_selecting_a_country_updates_the_chart():
    at = AppTest.from_file("app.py").run()
    at.selectbox[0].select("Country B").run()  # simulate a widget interaction, then rerun
    assert "Country B" in at.title[0].value
```

`AppTest.from_file(...)` loads and runs the script once (matching Streamlit's real "run the whole script" execution model, see [Core Concepts, §1](#)). The returned object exposes the rendered widget tree by type (`at.metric`, `at.selectbox`, `at.dataframe`, `at.warning`, and so on), each as a list in the order they appear on the page. Simulating a widget interaction (`.select(...)`, `.set_value(...)`, `.click()` depending on widget type) followed by `.run()` re-executes the script exactly as a real user interaction would, letting you assert on the *new* rendered state.

`at.exception` is worth checking in almost every test — an unhandled exception during a script run is exactly the kind of regression this tooling exists to catch, distinct from (and often more informative than) a specific assertion failure elsewhere.

2. Testing fixture data, not production data

Exactly as with API testing ([Python API Backend, §4](#)), point the app's loaders at small, controlled fixture data rather than real pipeline output — write a small CSV to a temporary path and monkeypatch the loader's source path before running `AppTest`, so tests don't depend on (or need to regenerate) production-scale data.

3. A candid note on how much automated coverage is typical in practice

`AppTest` is a genuinely useful tool, worth reaching for once a Streamlit app grows past a quick prototype. That said, it's common in practice for small, internal Streamlit dashboards to be verified primarily through **manual checking** — run the app, click through each page, confirm each renders (or shows the expected "not ready" message from [Core Concepts, §7](#)) — rather than a full automated suite. This is a reasonable trade-off for a tool whose whole value proposition is minimizing the engineering investment needed to get something working; the same reasoning that makes Streamlit attractive for rapid dashboarding also tends to make a lighter testing footprint an acceptable choice for it, at least until the app grows complex or important enough that the cost of a manual click-through regression check exceeds the cost of writing `AppTest` coverage.

4. A minimal manual verification checklist

Where automated coverage isn't (yet) in place, a lightweight, repeatable manual checklist still catches most real regressions:

- Every page loads without an unhandled exception.
- Every page's "data not available" message (if applicable) appears correctly when its underlying data file is absent, and disappears once it's present.
- Every interactive widget (selectbox, multiselect, slider) actually changes what's displayed when exercised.
- Charts render with the expected title, axis labels, and legend (per [Charting & Visualization](#)).

5. Core tools summary

Tool	Used for
<code>streamlit.testing.v1.AppTest</code>	headless, automated testing of a Streamlit script's rendered output
<code>pytest</code>	the test runner <code>AppTest</code> -based tests are typically written with

See also

- [Streamlit index](#)
- [Core Concepts](#) — the widgets and data-loading patterns being tested
- [Deployment](#) — running the tested app in a real environment